NATURAL LANGUAGE PROCESSING
FIRST MID SESSIONAL – EXTENDED MODEL ANSWER SHEET

B.Tech. Seventh Semester – Information Technology | Elective-V

Q.1 (a) Text Preprocessing and Normalization

Text preprocessing is the process of converting raw, unstructured text into a clean and standardized form that can be understood and processed by an NLP system. It reduces noise and makes important linguistic information easier for algorithms to learn. The major steps are:

1. Tokenization

Tokenization divides a sentence or document into smaller units called tokens. Tokens may be words, punctuation marks, or subwords. For example, the sentence “The students are studying NLP.” can be tokenized as [The, students, are, studying, NLP, .]. Tokenization is the basic step used before most NLP operations.

2. Stop-word Removal

Stop words are very common words such as “the”, “is”, “are”, “a”, “an”, “of”, and “to”. They often carry little information for tasks such as document classification. Removing them reduces the number of features. For example, “The boy is playing in the park” may become “boy playing park”. However, stop-word removal should be used carefully because words such as “not” can be important for sentiment analysis.

3. Stemming

Stemming reduces related words to a common stem by applying simple rules, often by removing prefixes or suffixes. For example, “playing”, “played”, and “plays” may be reduced to “play”. Stemming is fast but the resulting stem may not always be a valid dictionary word; for example, “studies” may become “studi” with some stemmers.

4. Lemmatization

Lemmatization converts a word to its meaningful dictionary base form, called a lemma, using vocabulary and grammatical information. For example, “studies” becomes “study”, “better” may be mapped to “good” depending on the linguistic resources, and “running” becomes “run”. It is generally more linguistically accurate than stemming but can be slower.

Conclusion: Example of complete preprocessing: Raw sentence: “The students are studying Natural Language Processing!” After tokenization: [The, students, are, studying, Natural, Language, Processing, !] After stop-word removal: [students, studying, Natural, Language, Processing] After stemming: [student, studi, natur, languag, process] After lemmatization: [student, study, Natural, Language, Processing] Thus, preprocessing and normalization improve consistency, reduce unnecessary vocabulary, and prepare text for tasks such as classification, sentiment analysis, information retrieval, and machine translation.

Q.1 (b) Concept of Text Preprocessing

Text preprocessing is the preparation of raw text before it is supplied to an NLP algorithm. Real-world text may contain punctuation, unnecessary spaces, spelling variations, numbers, symbols, URLs, stop words, and different forms of the same word. Preprocessing converts this noisy input into a useful representation.

Important preprocessing operations

Common operations include:

• Cleaning unwanted characters, HTML tags, URLs, and extra spaces.

• Converting text to a consistent case, usually lowercase.

• Tokenization into words or subwords.

• Stop-word removal when appropriate.

• Stemming or lemmatization to normalize word forms.

• Handling punctuation, numbers, emojis, and special symbols according to the application.

• Converting the cleaned text into numerical features such as Bag-of-Words or TF-IDF.

Example

Consider: “NLP is AMAZING!!! NLP helps computers understand human language.” A basic preprocessing pipeline may produce: “nlp amazing nlp helps computers understand human language”. It can then be tokenized into individual words and represented numerically for machine learning.

Importance

Preprocessing reduces noise, decreases vocabulary size, improves computational efficiency, and can improve model performance. The exact operations should depend on the NLP task; aggressive preprocessing may remove useful information in applications such as sentiment analysis or named entity recognition.

Conclusion: In short, text preprocessing creates a standardized input representation from raw language. It is an essential first stage in many NLP pipelines because machine-learning models generally work better when irrelevant variation and noise are controlled.

Q.2 (a) Natural Language Processing: Scope and Applications

Natural Language Processing (NLP) is a branch of Artificial Intelligence, computer science, and linguistics concerned with enabling computers to process, understand, interpret, and generate human language. NLP deals with both written and spoken language.

Scope of NLP

The scope of NLP includes:

1. Text preprocessing and normalization.
2. Morphological and syntactic analysis such as part-of-speech tagging and parsing.
3. Semantic analysis and word-sense understanding.
4. Information extraction and named entity recognition.
5. Text classification and sentiment analysis.
6. Question answering and conversational systems.
7. Machine translation.
8. Speech recognition and language generation.
9. Text summarization and information retrieval.

1. Machine Translation

Machine translation automatically converts text from one language to another. For example, an English sentence can be translated into Hindi or Marathi. Modern systems use neural networks and contextual representations to improve fluency and meaning preservation.

2. Sentiment Analysis

Sentiment analysis identifies the opinion or emotional polarity expressed in text, such as positive, negative, or neutral. It is used for product reviews, social-media monitoring, customer feedback, and market research.

3. Chatbots

NLP enables chatbots to understand user questions and generate suitable responses. Applications include customer support, educational assistants, banking helpdesks, and service portals. Intent detection and entity extraction are commonly used in such systems.

4. Speech Recognition

Speech recognition converts spoken language into text. It is used in voice assistants, dictation systems, call-center analytics, and accessibility tools. The system must handle pronunciation, accents, background noise, and context.

Conclusion: Therefore, NLP connects human communication with computer systems. Its applications improve automation, search, communication, accessibility, and decision support across many industries.

Q.2 (b) Linguistics

Linguistics is the scientific study of human language. It examines how language is structured, how meaning is created, how sounds are produced, and how language is used in society. Linguistics is an important foundation of NLP because computers need linguistic knowledge to process human language effectively.

Major areas of linguistics

1. Phonetics: studies the physical production and perception of speech sounds.

2. Phonology: studies how sounds are organized and function within a language.

3. Morphology: studies the internal structure of words and morphemes. For example, “unhappiness” contains meaningful parts such as “un-”, “happy”, and “-ness”.

4. Syntax: studies how words combine to form grammatical sentences.

5. Semantics: studies literal meaning of words, phrases, and sentences.

6. Pragmatics: studies meaning in context and how speakers convey intended meaning.

7. Discourse: studies relationships between sentences and larger units of communication.

Linguistics in NLP

Linguistic knowledge supports tasks such as tokenization, part-of-speech tagging, parsing, semantic analysis, machine translation, speech processing, and information extraction. For example, syntax helps a system identify the grammatical relationship between “student” and “reads” in “The student reads a book.” Semantics helps distinguish meanings of ambiguous words using context.

Example

The word “unhappiness” can be analyzed morphologically as un + happy + ness. In the sentence “The student opened the book,” syntax identifies the subject, verb, and object, while semantics helps interpret their roles.

Conclusion: Thus, linguistics provides the theoretical understanding of language, while NLP uses computational methods to implement language processing on computers. Both fields complement each other in building effective language technologies.

Q.3 (a) Text Categorization

Text categorization, also called text classification, is the process of assigning a document or text to one or more predefined categories. It is a supervised machine-learning task in which a model learns from labeled examples and predicts the category of unseen text.

Basic steps

1. Collect labeled documents, such as emails labeled Spam or Not Spam.
2. Preprocess the text using tokenization, normalization, and other suitable techniques.
3. Convert text into numerical features using Bag-of-Words, TF-IDF, or embeddings.
4. Split data into training and testing sets.
5. Train a classifier such as Naive Bayes, Decision Tree, Logistic Regression, or SVM.
6. Predict categories for new documents.
7. Evaluate the model using accuracy, precision, recall, F1-score, or a confusion matrix.

Example

Suppose an email classifier has two classes: Spam and Ham. Training examples containing terms such as “free”, “offer”, and “winner” may be associated with Spam, while ordinary messages about classes or meetings may be associated with Ham. For a new email, the trained model calculates the likelihood of each class and assigns the most suitable label.

Applications

Text categorization is used in spam detection, news classification, sentiment classification, topic labeling, customer-support routing, document organization, and toxic-content detection.

Advantages and challenges

It can automatically organize large collections of documents and reduce manual effort. Challenges include class imbalance, ambiguous language, domain-specific vocabulary, sarcasm, spelling variations, and changes in language over time.

Conclusion: Hence, text categorization converts unstructured language into useful predefined labels. The quality of preprocessing, feature representation, training data, and classifier all affect the final classification performance.

Q.3 (b) Various N-gram Models

An n-gram is a contiguous sequence of n words or tokens in a text. N-gram language models estimate the probability of a word based on a limited history of previous words. They are useful in language modeling, spelling correction, autocomplete, speech recognition, and other NLP tasks.

1. Unigram model

A unigram model considers one word at a time and assumes words are independent. For a sentence containing w1, w2, …, wn, the probability is approximated as the product of individual word probabilities. Example tokens: “I”, “like”, “NLP” are treated independently.

2. Bigram model

A bigram contains two consecutive words. It estimates P(wi | wi-1). For “I like NLP”, the bigrams are “I like” and “like NLP”. Bigrams capture immediate word-to-word relationships better than unigrams.

3. Trigram model

A trigram contains three consecutive words and estimates P(wi | wi-2, wi-1). For “I like studying NLP”, examples include “I like studying” and “like studying NLP”. It uses more context than a bigram.

4. Higher-order n-grams

4-gram, 5-gram, and larger models use longer contexts. They can capture more local context but require more training data and have a greater risk of unseen combinations and data sparsity.

Probability and smoothing

The general chain approximation is P(w1…wn) ≈ $\prod$ P(wi | previous n–1 words). Because many possible n-grams may never occur in training data, smoothing methods such as Laplace/add-one smoothing can assign non-zero probability to unseen n-grams. Larger n usually gives more context but increases computational and data requirements.

Conclusion: Example: Sentence: “I love natural language processing.” Unigrams: I | love | natural | language | processing Bigrams: I love | love natural | natural language | language processing Trigrams: I love natural | love natural language | natural language processing Thus, n-gram models provide a simple statistical way to model local word sequences. Although neural language models are widely used today, n-grams remain important for understanding the fundamentals of language modeling.

Q.4 (a) Naive Bayes Algorithm for Text Classification

Naive Bayes is a probabilistic supervised-learning algorithm widely used for text classification. It is based on Bayes' theorem and makes a simplifying assumption that features are conditionally independent given the class. Despite this assumption, it often performs well for high-dimensional text data.

Bayes' theorem

$P(C|D) = [P(D|C)\ P(C)] / P(D)$

where C is a class and D is a document. For a document containing words w1, w2, …, wn, the Naive Bayes approximation is:

$P(C|D) \propto P(C) \times \prod P(wi|C)$

The predicted class is the class with the highest posterior score.

Working steps

1. Collect labeled training documents.
2. Calculate the prior probability P(C) for each class.
3. Calculate word probabilities P(w|C) from the training documents.
4. Apply smoothing, commonly Laplace smoothing, so unseen words do not produce zero probability.
5. For a new document, multiply the class prior by the word likelihoods.
6. Assign the document to the class with the highest posterior score.

Worked example

Suppose there are two classes: Sports and Technology. Assume a small training set gives:

P(Sports)=0.6, P(Technology)=0.4.

For the words "team" and "software", suppose after Laplace smoothing:

P(team|Sports)=0.20, P(software|Sports)=0.05

P(team|Technology)=0.05, P(software|Technology)=0.20

For D = "team software":

Sports score = 0.6 × 0.20 × 0.05 = 0.006

Technology score = 0.4 × 0.05 × 0.20 = 0.004

Since 0.006 > 0.004, the document is classified as Sports. These values are unnormalized scores; normalization is unnecessary when only the winning class is required.

Advantages

Naive Bayes is simple, fast, memory-efficient, easy to implement, and effective for sparse high-dimensional text. It works well even with relatively small training datasets.

Limitations

The conditional-independence assumption is often unrealistic because words are related to one another. Performance can decrease when word order and complex interactions are important. Zero-frequency problems must also be handled with smoothing.

Conclusion: Naive Bayes is especially useful as a strong baseline for spam detection, topic classification, sentiment analysis, and document classification because text data usually contains many sparse features.

Q.4 (b) Decision Tree vs Support Vector Machine

Decision Tree and Support Vector Machine (SVM) are supervised-learning algorithms that can be used for text classification. Their working principles are different, and each has particular advantages and limitations.

Decision Tree – working principle

A Decision Tree recursively splits the training data using feature-based rules. Each internal node represents a decision on a feature, branches represent outcomes, and leaf nodes represent class labels. Splits are commonly selected using measures such as Gini impurity or information gain. For text classification, features may be word counts or TF-IDF values.

SVM – working principle

SVM finds a decision boundary, or hyperplane, that separates classes while maximizing the margin between the classes. The most informative training examples near the boundary are called support vectors. Kernels can be used when a nonlinear boundary is required, although linear SVM is particularly effective for high-dimensional sparse text.

Comparison

Decision Tree:

• Easy to understand and visualize.

• Produces rule-based decisions.

• Requires relatively little feature scaling.

• Can overfit unless depth and other parameters are controlled.

• May be less effective than linear SVM on very high-dimensional sparse text.

SVM:

• Strong performance in high-dimensional spaces.

• Linear SVM works very well with sparse TF-IDF features.

• Good generalization with an appropriate margin and regularization.

• Less interpretable than a small Decision Tree.

• Training can be more computationally demanding for very large datasets, depending on implementation.

Suitability for text data

Text classification commonly produces thousands or millions of sparse features. Linear SVM is often highly suitable because it can learn an effective separating hyperplane in high-dimensional space. Decision Trees can still be useful when interpretability and rule-based explanations are important, but a single tree may struggle with extremely large sparse vocabularies.

Conclusion: Conclusion: If interpretability and simple decision rules are the priority, a Decision Tree is attractive. If the main goal is strong classification performance on high-dimensional sparse text such as TF-IDF vectors, SVM—especially a linear SVM—is generally a more suitable choice.